

Código de asignatura:

Tipo de asignatura	☐ Obligatoria ☒ Electiva
Número de créditos ECTS	
Director o contacto del programa	
Coordinador o contacto de la asignatura	

Descripción y contenidos**1. Breve descripción**

Esta asignatura introduce a los estudiantes del programa de Tecnología en Desarrollo de Software a los fundamentos y prácticas de la cultura DevOps, la automatización de procesos de desarrollo y operaciones, la integración continua, el despliegue continuo, el uso de contenedores y el monitoreo básico. Su enfoque es teórico-práctico y está alineado con las necesidades actuales de la industria del software, privilegiando la aplicación operativa sobre la teoría avanzada, acorde con el perfil de formación tecnológica.

2. Objetivos

Al finalizar la asignatura, el estudiante será capaz de diseñar, implementar y documentar un flujo automatizado de construcción, prueba y despliegue de una aplicación de software sencilla, empleando herramientas de integración continua, contenedores y monitoreo básico, y participando activamente en un equipo de trabajo bajo la filosofía DevOps.

3. Resultados de aprendizaje

RA1: Identifica el diseño, la implementación y la socialización de proyectos de desarrollo de software como una respuesta a problemáticas relacionados con la productividad en el manejo de la información actual y futura de la empresa, y de la sociedad en condiciones de responsabilidad, eticidad y equidad (COMPRENDE)

RA2: Implementa estándares internacionales de calidad en la producción de software en cada una de sus etapas. (HACE)

RA3: Propone proyectos empresariales de desarrollo de software evidenciando su factibilidad técnica, financiera y comercial. (SABE)

RA4: Participa en equipos de trabajo en la producción de software en ambientes presenciales o remotos de manera colaborativa en una lengua diferente de la materna. (HACE - SABE)

4. Contenido

T1: Fundamentos DevOps.

☐ *Principios y evolución de la cultura DevOps: Se exploran los fundamentos históricos y filosóficos que dieron origen a DevOps como una respuesta a la separación tradicional entre desarrollo y operaciones. Se analizan las prácticas claves: colaboración, automatización, integración y retroalimentación continua.*

☐ *Ciclo de vida DevOps: Revisión general de las fases de planeación, desarrollo, integración, pruebas, entrega, monitoreo y retroalimentación. Se presenta el modelo CALMS (Culture, Automation, Lean, Measurement, Sharing).*

T2: Introducción a git.

☐ *Introducción a los conceptos básicos de Git, configuración de repositorios locales y remotos, y comandos fundamentales para el flujo de trabajo: init, clone, add, commit, push, pull.*

☐ *Ramificación y trabajo colaborativo: Introducción a la gestión de ramas, fusión de código (merge), resolución de conflictos, y buenas prácticas para trabajo en equipo. Simulación de una dinámica de colaboración en un proyecto con múltiples contribuidores.*

T3: Fundamentos de Linux.

- *Comandos esenciales de terminal y manipulación de archivos: Navegación de la estructura de directorios, permisos, ejecución de procesos y edición básica. Uso de comandos como ls, cd, grep, ps, chmod, etc.*
- *Automatización con scripts de shell: Creación de scripts sencillos para automatizar tareas repetitivas como backups, despliegue o configuración de entorno*

T4: Contenedores: fundamentos y orquestación (Docker Compose).

- *Creación y gestión de contenedores: Explicación de imágenes, contenedores, red interna y almacenamiento persistente. Construcción de imágenes personalizadas usando Dockerfile, gestión de versiones, y uso de Docker Hub.*
- *Orquestación de servicios con Docker Compose: Creación de entornos multi-contenedor (frontend, backend, base de datos), configuración de dependencias entre servicios y uso de volúmenes compartidos.*

T5: Integración continua y despliegue continuo.

- *Pipelines CI/CD y automatización de pruebas: Introducción al concepto de integración continua y automatización de pruebas unitarias. Configuración de un pipeline usando GitHub Actions o GitLab CI para compilar, testear y validar código.*
- *Despliegue automático y entornos de prueba: Configuración de despliegue continuo en servidores o servicios en la nube (Heroku, Vercel, Render). Manejo de variables de entorno, versionamiento de infraestructura y buenas prácticas de seguridad en producción.*

T6: Monitoreo, métricas y logs

- *Observabilidad: conceptos y herramientas básicas: Introducción a la observabilidad como práctica integral. Recolección de métricas con Prometheus y visualización mediante Grafana. Logs centralizados con herramientas como Loki o ELK.*
- *Implementación de alertas básicas y análisis de comportamiento: Definición de umbrales, creación de reglas simples de alerta y revisión de escenarios comunes de fallas. Ejercicio de laboratorio con métricas generadas por contenedores activos.*

Competencias: Capacidad de comprender los fundamentos de la cultura DevOps y los principios de integración y entrega continua. Será competente para construir pipelines básicos, utilizar contenedores Docker y aplicar nociones de infraestructura automatizada y monitoreo elemental. Además, fortalecerá habilidades colaborativas dentro de equipos de desarrollo y operaciones, demostrará adaptabilidad frente a herramientas tecnológicas actuales y comunicará de manera clara y organizada los procedimientos técnicos que implemente.

5. Recursos

- *Humble, J., & Farley, D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.*
 - *Kim, G., Debois, P., Willis, J., & Humble, J. The DevOps Handbook.*
 - *Burns, B., Beda, J., & Hightower, K. Kubernetes: Up & Running (capítulos seleccionados).*
- Bibliografía complementaria:*
- *Fowler, M. Refactoring (capítulos seleccionados).*
 - *Documentación oficial de Git, GitHub Actions o GitLab CI.*
 - *Documentación oficial de Docker y Docker Compose.*

Herramientas de Software.

- *Termius o MobaXterm.*
- *VPS o WSL*
- *VS code*

6. Herramientas técnicas de soporte para la enseñanza

- *Presentaciones conceptuales.*
- *Exposiciones y discusiones en clase.*
- *Lecturas sugeridas.*
- *Ejercicios propuestos.*

7. Trabajos en laboratorio y proyectos

• **Taller 1 – Orquestación de servicios con Docker Compose**

El estudiante creará un entorno de ejecución completo utilizando contenedores para frontend, backend y base de datos. Aprenderá a definir servicios en docker-compose.yml, manejar volúmenes y redes internas, e integrar variables de entorno. Se evaluará el despliegue exitoso del sistema.

• **Taller 2 – Automatización de flujo CI básico con GitHub Actions**

El estudiante implementará un pipeline de integración continua que compile y realice pruebas automáticas sobre una aplicación sencilla. Aprenderá a definir workflows en YAML, configurar acciones y usar condiciones de ejecución. Este taller refuerza los conceptos de CI, versionamiento y automatización desde Git.

• **Proyecto Final – Flujo DevOps completo para una aplicación funcional**

En equipos, los estudiantes desarrollarán un flujo DevOps automatizado para una aplicación realista (puede ser web o API). El proyecto debe incluir: control de versiones, automatización CI/CD, contenedores dockerizados, despliegue en entorno de prueba y monitoreo básico. Se valorará la documentación, presentación técnica y aplicación de buenas prácticas.

8. Métodos de aprendizaje

- *Clases magistrales.*
- *Lecturas de artículos especializados.*
- *Videos explicativos y ayudas visuales.*
- *Talleres prácticos.*
- *Tutorías.*

9. Métodos de evaluación

La evaluación será continua y formativa, con los siguientes porcentajes sugeridos por corte:

- *Fundamentos de DevOps y manejo de GIT: 35%. (RA1, RA2, RA4)*
- *Fundamentos de Linux y Conceptos de Docker: 30% (RA2, RA3, RA4)*
- *Integración y despliegue continuo, monitoreo: 35% (RA2, RA4)*

Para aprobar la asignatura el estudiante deberá cumplir con los requisitos de asistencia establecidos por la Universidad Tecnológica de Pereira y obtener la nota mínima aprobatoria según el reglamento estudiantil vigente.